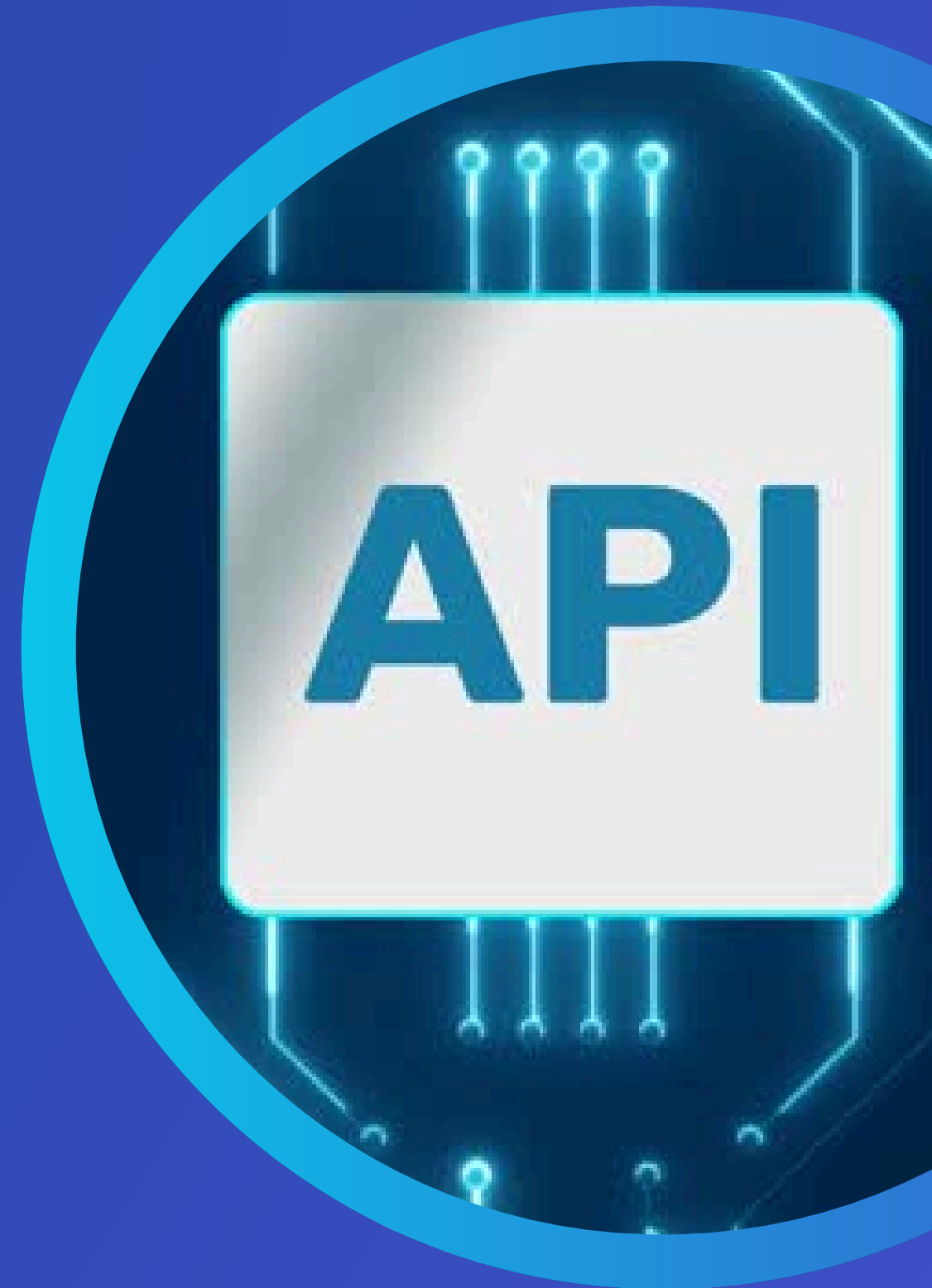


API Simplon Gestion de formations

Flora, Fatima, Edilene

SIMPLON
.CO

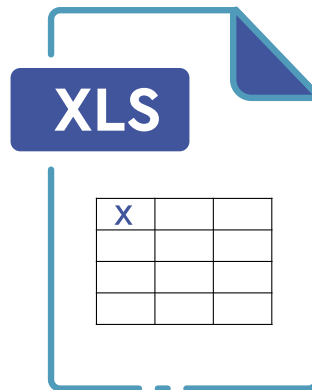


Contexte



Problématique client

Digitaliser la gestion de ses apprenants, formateurs et sessions de formation



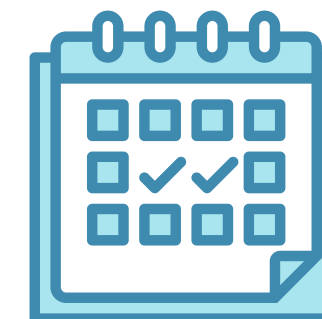
Inscriptions manuelles

Inscriptions non centralisées, via des tableurs



Suivi dispersé

Pas de vision d'ensemble sur les formations et apprenants



Gestion difficile

Planification des sessions sans outil dédié

Technologies utilisées:

SQLAlchemy v2

ORM et modèles

FastAPI

Framework web

Pydantic

DTO et validation

Alembic

Migration base de données

SQLite

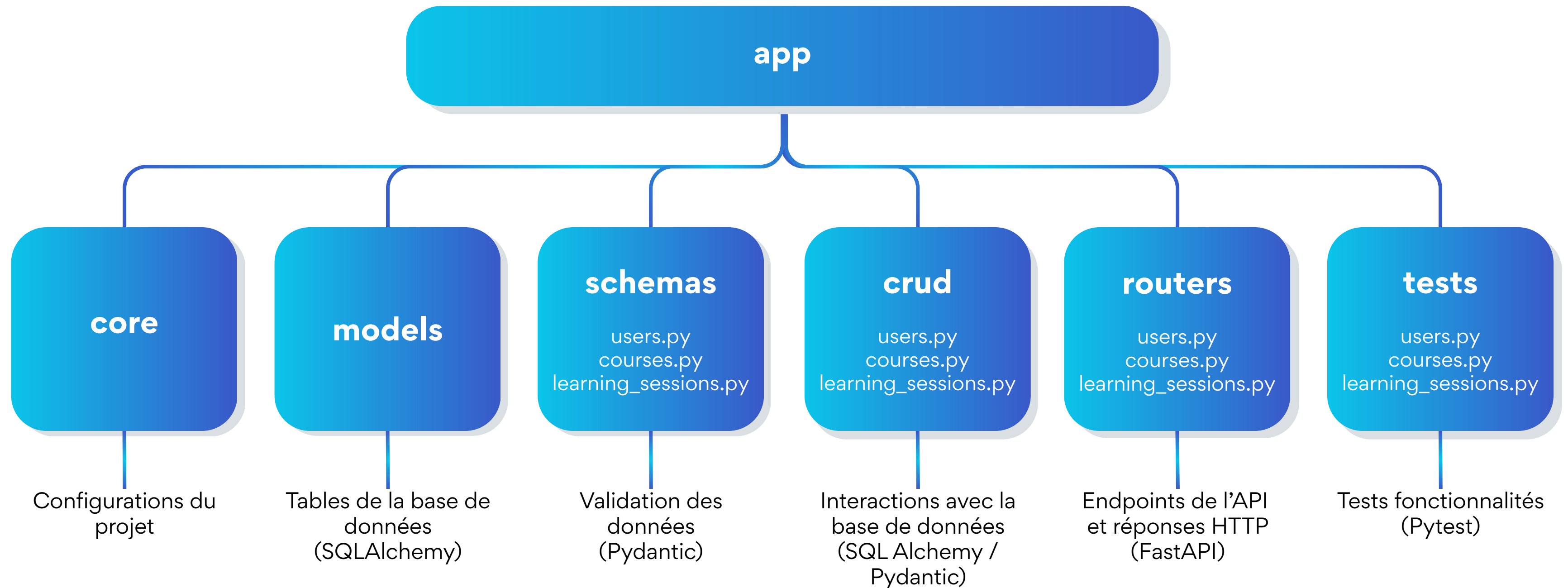
Base de données

Pytest

Tests des fonctionnalités

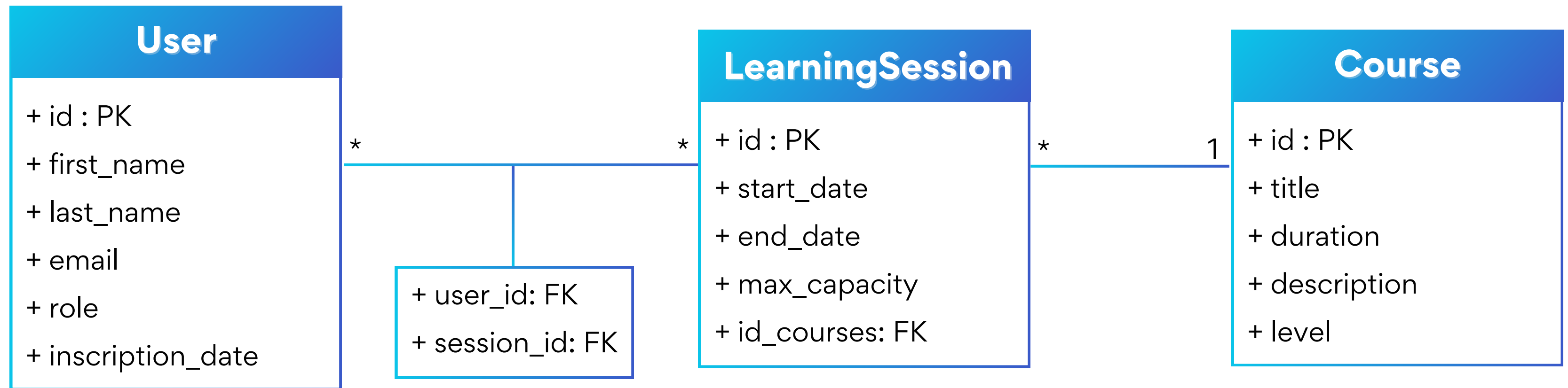
Architecture du projet

Séparation des principes



UML

Diagramme de classes



Modèles SQLAlchemy

User

- + id : int
- + first_name : str
- + last_name : str
- + role : Enum ("administrateur", "formateur", "apprenant")
- + inscription-date: date
- + deleted_at: datetime
- + learning_sessions :
List["Learning_session"]

Inscription

- + user_id : int
- + session_id : int

LearningSession

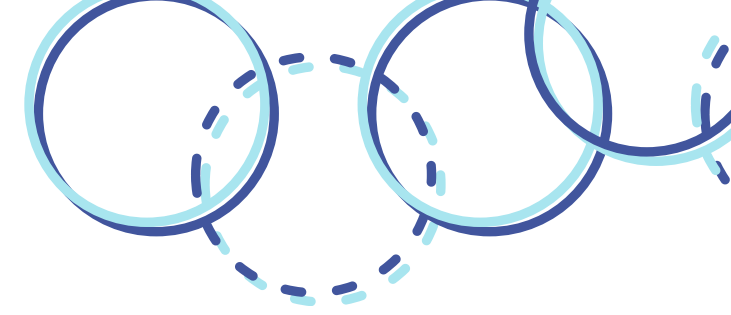
- + id : int
- + start_date : date
- + last_date : date
- + max_capacity : int
- + course_id : int
- + deleted_at : datetime
- + course : Course
- + user : List["User"]

Course

- + id : int
- + title : str
- + duration : int
- + description : str
- + level : str
- + deleted_at : datetime
- + learning_session :
List["LearningSession"]

DTOs & Validation des données

Schémas (Pydantic V2)



Schémas DTO par entité

XxCreate

Données reçues à la création

XxUpdate

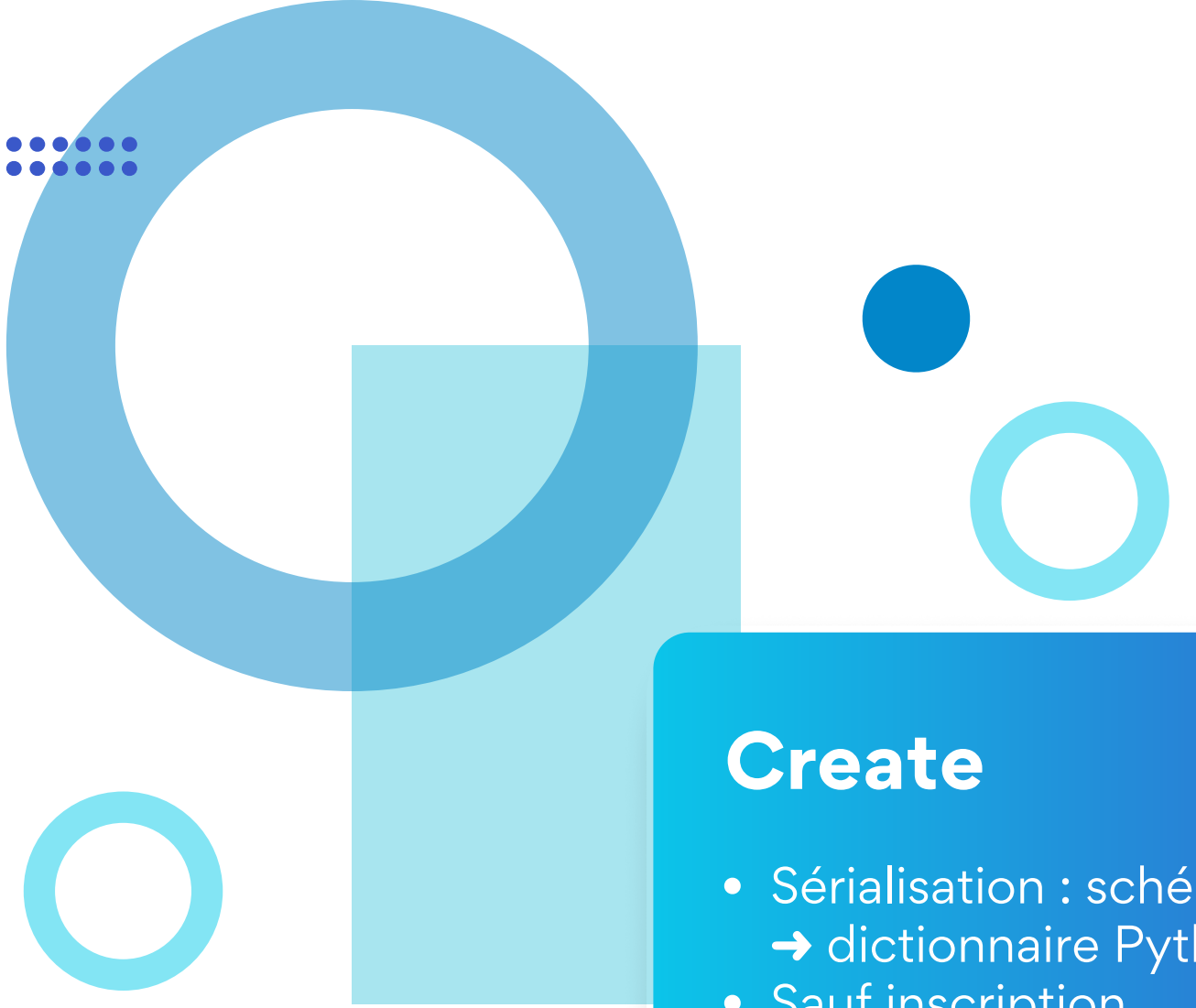
Champs optionnels (Patch)

XxResponse

Données retournées à l'API

Règles de validation

- Email valide et unique (EmailStr)
- Durée > 0 (Field gt=0)
- Capacité ≥ 1 et ≤ 50 , Field (ge=1, le=50)
- end_date > start_date (@model_validator)
- Titre non vide (@field_validator)
- Rôle et niveau via Enum



CRUD

Interactions avec la base de données

Create

- Sériàlisation : schéma Pydantic
→ dictionnaire Python
- Sauf inscription

Read

- Get all / get one
- Liens entre entités

Update

- Champs modifiés uniquement
- Learning Sessions : vérification des dates

Delete

- Hard delete si pas de données associées
- Sinon soft delete : deleted at

Routeurs

Endpoints API

Utilisateur

GET	/users/	Read Users
POST	/users/	Create User
GET	/users/{user_id}	Read User By Id
PATCH	/users/{user_id}	Update User
DELETE	/users/{user_id}	Delete User
DELETE	/users/{user_id}/session/{session_id}	Delete Inscription
POST	/users/{user_id}/session/{session_id}	Inscription
GET	/users/{user_id}/sessions	Get Sessions

Formation

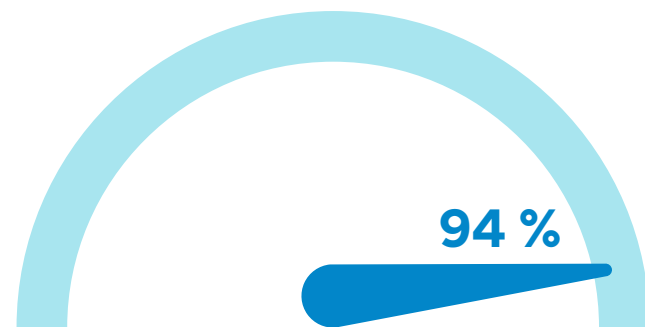
POST	/courses/	Create
GET	/courses/	List All
GET	/courses/{course_id}	Get One
PATCH	/courses/{course_id}	Update
DELETE	/courses/{course_id}	Delete
GET	/courses/{course_id}/sessions	Get Learning Sessions

Session

GET	/learning-sessions/	Get Learning Sessions
POST	/learning-sessions/	Create Learning Session
GET	/learning-sessions/{learning_session_id}	Get Learning Session
PATCH	/learning-sessions/{learning_session_id}	Update Learning Sessions
DELETE	/learning-sessions/{learning_session_id}	Delete Learning Session
GET	/learning-sessions/{learning_session_id}/users	Get Learning Se

Tests des fonctionnalités

Pytest



Configuration

- Base de données test
- Connexion réinitialisée à chaque test
- Simulation de requêtes HTTP

Tests unitaires

- Validations Pydantic fonctionnelles
- Interactions avec la base de données

Tests d'intégration

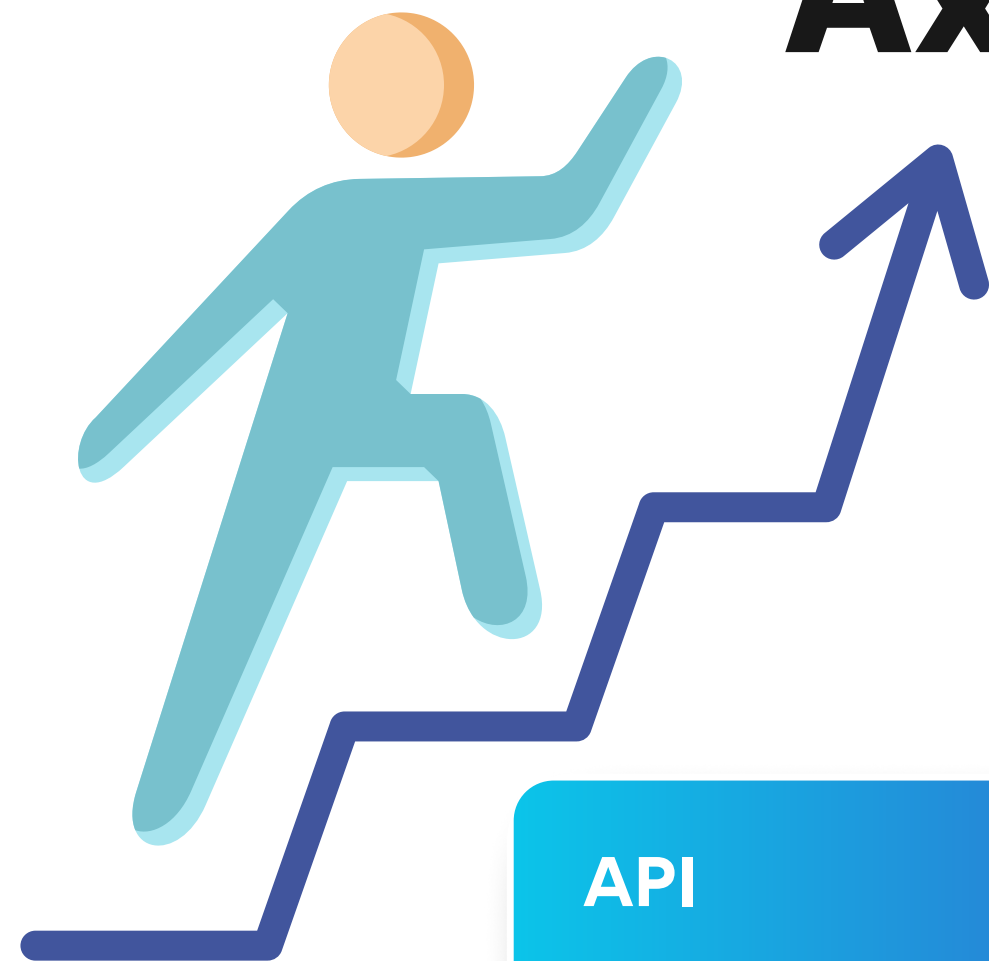
Parcours complet :

- Appel d'un endpoint
- Validation de données
- CRUD
- Réponse (contenu + statut HTTP)



Démo

Axes d'amélioration



API

- Recherche filtrée
- Pagination avancée
- Tri des résultats
- Statut sessions de formation

Métier

- Listes d'attente
- Statistiques simples :
 - taux de remplissage
 - nb apprenants par formation

Technique

- Login sécurisé
- Migration vers PostgreSQL
- Autorisations par rôle

+

**Merci pour votre
attention !**

Des questions ?

